Group 60 Progress Report:
2048 Machine learning model to determine the next best move

Huaihui Zeng, Suzanne Abdullah
{zengh22,abduls61}@mcmaster.ca

1 Introduction

We used an existing algorithm to play 2048 and collected the board state before each recorded move and the move it chose as training data. We trained the model on these data without explicitly giving it the algorithm's rules. We aim to find out whether the model can learn strategies and imitate the algorithm using only examples of board states and chosen moves.

2 Related Work

Machine learning models that aim to select moves in 2048 resulting in optimal moves are not uncommon, and many have been created prior to ours. Our biggest reference for this project was the github repository the project used for data collection (game difficulty et al., 2024). This system used a deep-knowledge machine learning model to create their algorithms. Our key takeaway from the strategy used in this repository was that a lack of monotonicity in the model can help greatly in creating a model that can surpass other 2048 AIs, as the engines flexibility allows it to fix dynamically fix mistakes. This repository used a reinforcement learning model, which made sense considering the nature of the problem. The penalty for reaching death states shifts dynamically in this model, which might be a good strategy for future iterations fo the model. This model reached very high levels of success, reaching a 100% success rate over 8k moves. Two other models (Chan, 2022) (Fuchs) were first created using a random direction choice, and then was rewarded in increasingly complex manors depending on the models progress in the game. These models also used a neurel network strategy, but on a much simpler level than the github repository, as only one hidden layer was used during this implementation. This article by Fuchs greatly informed our method of feature classification, as we used a similar framework in storing moves to the one described here. This article also uniquely introduces a 'penalty' for illegal moves, which could be useful in trying to filter out unnecessary behaviour in the model early on. A dissertation by Dr. Guei (Guei, 2023) breaks down multiple strategies that can be employed for a reinforcement learning approach to the game of 2048. This article provides a lot of details about the results of multiple strategies for training a 2048, including ensemble learning, Monte Carlo tree search and deep reinforcement learning. This article is very intensive, and will be very useful to our team moving forward with different strategies during the rest of the project. The investigations into 'greedy' algorithms was especially interesting, as our team had yset to consider their application potential in the project. Finally, an article from 2016 (Nie et al., 2016) was investigated, which used a minimax and expectimax search tree strategy, that were improved upon via human heuristics. This paper also used reinforcement learning, to some degree of success after pruning, which showed our team that a tree strategy could be fruitful in future implementations. Overall, our biggest takeaways from our research were that reinforcement strategies were the most effective, and larger datasets were more ideal in getting more accurate results.

3 Dataset

Our dataset was collected using a scraping technique to generate games from the front-end implementation of the existing 2048 ML model our program utilizes: https://game-difficulty.github.io/2048EndgameTablebase/. Using Selenium and Beautifulsoup libraries, our team created a program that would open the 2048 simulator in a browser window and increment through games one turn at a time, saving the board state and current score on each move. The boardstate was collected from the corresponding hexadecimal code

TileA1	0	0	0	0	0	0
TileA2	0	0	0	0	0	0
TileA3	0	0	0	1	0	2
TileA4	0	0	0	0	1	2
TileB1	0	2	2	0	0	0
TileB2	0	0	1	0	0	0
TileB3	1	0	0	2	2	1
TileB4	0	0	0	1	1	2
TileC1	0	1	1	0	0	0
TileC2	0	0	0	0	0	0
TileC3	0	0	0	0	1	0
TileC4	1	0	0	1	1	0
TileD1	0	0	1	0	0	0
TileD2	0	0	0	0	0	0
TileD3	0	0	0	0	0	0
TileD4	0	1	0	1	1	1
Move	0	RIGHT	LEFT	RIGHT	RIGHT	UP
Game_Number	1	1	1	1	1	1
Game_Turn	1	2	3	4	5	6
Game_Score	0	4	4	4	4	12

Table 1: Sample of 5 points in 2048 dataset

given at the top of the page. This code was split into individual board tiles (1 digit for each tile), and each tile was converted into its corresponding decimal value. Each board tile was stored in the logarithmic form (e.g. 4 would be stored as 2). Each board tiles name was adherent to the A-D:1-4 formatting, with the letter representing the row, and the number representing the column. At the end of a game, the board would be cleared and reset to a valid starting state that was randomly generated by the program. The dataset also includes an estimate for what move was made, based off of the board state before and after each move was made. When a valid estimate for the previous move could not be made, the program defaulted to a 0 value for the move value. These zeroes were filtered out in the preprocessing stage. The program exports collected data to an excel sheet, and multiple datasheets were created with varying amounts of games and turns. A sample of the one of the generated dataset has been provided above.

4 Features

We represented each 4×4 board as 16 numbers, one for each cell, in row order. An empty cell was encoded as 0, and a tile with value 2^n was encoded as n. We kept all 16 features and did not use data augmentation. We fitted `StandardScaler` on the training data and used it to standardize both the training and test data before running k-NN.

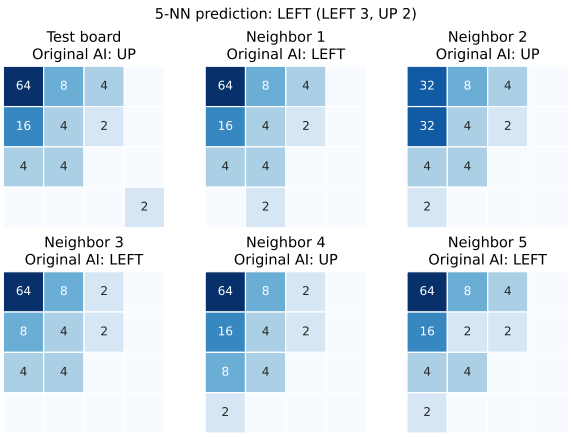


Figure 1: boards of the test data and 5 neighbors. 3 neighbors chose LEFT and 2 chose UP. So k-NN chose LEFT, while the original AI chose UP

5 Implementation (DONE)

We used scikit-learn to implement a k-nearest neighbors (k-NN)classifier with $k = 5$. The model takes the 16 encoded tile values from a board and predicts one of four moves: up, down, left, or right. We calculated the scaling statistics from the training boards and used them to standardize both the training and test boards.

For each test board, the classifier finds the five closest training boards using Euclidean distance. Each neighbor gets one vote, and the move with the most votes is the prediction. Our k-NN model stores the training boards and predicts by voting among the five closest ones, so it does not need a loss function during training.

For the baseline, we ranked the four moves by how often they appeared in the training data. For each test board, the baseline checks the moves in that order and picks the first one that slides or merges a tile.

6 Results and Evaluation

We split the data by game, using the first 80% of games for training and the remaining 20% for testing. We did not use a separate validation set or cross-validation. Each game contributed 50 recorded moves. We measured test accuracy as the percentage of predictions that matched the original AI's moves. For each test board, the baseline chose the legal move that appeared most often in the training data.

As shown in Table 2, k-NN beat the baseline in four of the five experiments. In the 200-game experiment, k-NN achieved 52.0% accuracy, compared

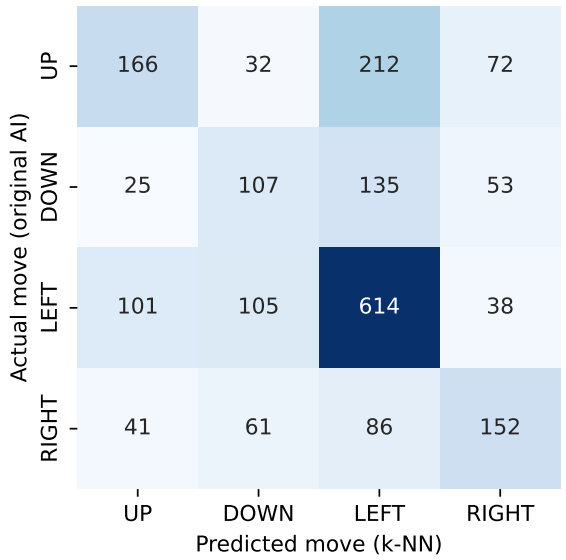


Figure 2: Confusion Matrix showing 52% accuracy

with 47.7% for the baseline.

Games	k-NN($k = 5$)	Baseline
5	48.0%	66.0%
10	49.0%	41.0%
50	54.8%	45.4%
135	52.4%	51.0%
200	52.0%	47.7%

Table 2: Move prediction accuracy by dataset size

7 Feedback and Plans

For the remainder of the project, our team aims to implement the SVM version of our Machine learning model, and compare it to our k-NN approach. This will provide us with a clear picture of the more applicable choice for this specific project. From the feedback on our proposal from Shreya, we were told that our project idea was interesting, and that our team had a good understanding of the problem, but we should further define more details about the implementation. Taking this advice, our team hopes to further develop our knowledge of the problem space going forward, and aim to do further research into ways to improve our existing model. We are also interested in investigating reinforcement learning approaches to building our model as well, as from our investigation into the related work, it seems like the most effective way to improve our model. Our team would also hope to test the model manually at some point, by trying moves generated by the algorithm to see how it

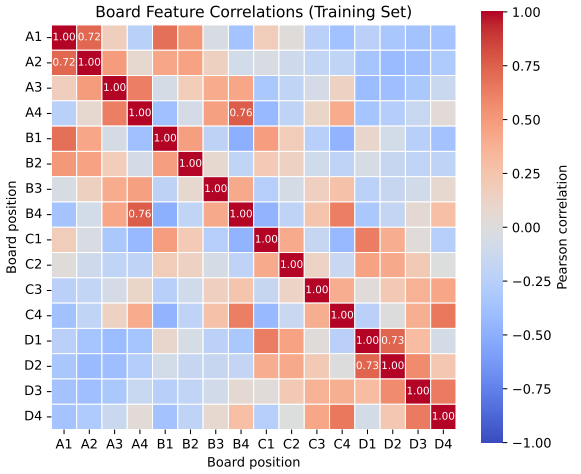


Figure 3: Heatmap, A4 (top-right cell) and B4 (cell below A4) are positively correlated across the training board. (r = 0.76)

compares to a human playing the game. Finally, we hope to improve the quality of the dataset, by looking into other data collection strategies that lead to more accurate results regarding the move label. This could be achieved by recording data from the repository directly instead of webscraping, which will be investigated in future iterations.

Team Contributions

Suzanne worked on the data collection program for the ML model, while Huaihui worked on the ML model itself. The sections of the report were split between team members. Huaihui completed the introduction, model inputs and implementation and evaluation strategies, while Suzanne completed the related work, dataset and preprocessing, TA feedback, team contribution sections, and formatting. Although the project was divided amongst group members, we each contributed input to eachothers sections and kept strong communication with eachother while working on the deliverable.

References

Lok Hin Chan. 2022. Playing 2048 with deep q-learning (with pytorch implementation).

Sam Fuchs. 2048 ai.

game difficulty, Narwhal, and Sitiyou. 2024. 2048endgametablebase.

Hung Guei. 2023. *On Reinforcement Learning for the Game of 2048*. Ph.D. thesis.

Yun Nie, Wenqi Hou, and Yicheng an. 2016. Ai plays 2048. *Stanford Edu*.